

PawTrack

Technical Overview & Theory of Operation

A pet-care platform connecting pet parents with vets, groomers, trainers and sitters — how it is built, and why it is built that way.

What this document is

A conceptual, theory-first walkthrough of the PawTrack codebase. It explains the architecture, the data model, and the key algorithms (booking concurrency, slot generation, reminders, real-time notifications, distance search) in plain language, with just enough code to anchor each idea. It is written so you can both understand the system and explain it to someone else — for a project review, viva, interview or onboarding.

Contents

1. The product in one page
2. Technology stack and why each piece was chosen
3. System architecture
4. Backend structure and the request lifecycle
5. Authentication and authorization
6. The data model
7. The booking engine (concurrency and state)
8. Availability: generating time slots
9. Notifications: scheduled reminders and real-time push
10. Provider discovery: geocoding and distance
11. Medical records and secure file handling
12. Reference data: species and breeds
13. Frontend architecture
14. Data integrity, auditing and security principles
15. API reference
16. Limitations, scaling path and future work
17. Explaining PawTrack: common questions and model answers

1. The product in one page

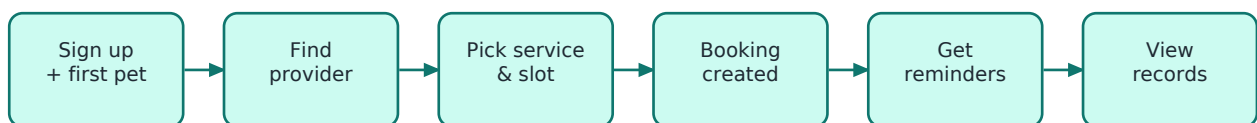
PawTrack is a two-sided web application. On one side are **pet parents**, who keep profiles for their pets, find nearby care providers, book appointments and keep a medical history. On the other side are **providers** — vet clinics, independent vets, groomers, trainers and pet sitters — who publish services, manage staff and their working hours, run their appointment calendar and upload records after each visit.

Everything in the system revolves around one central event: **a booking**. A booking ties together a pet, its owner, a provider, a service, a specific staff member and a specific time slot. Most of the interesting engineering — concurrency control, notifications, access control to medical records — exists to make that one event correct and useful for both sides.

Pet parent can...	Provider can...
Sign up with a first pet in one step	Sign up as one of five provider types
Add, edit and remove pets (age, weight, breed, alerts/allergies)	Create services (category, price, duration) and activate/deactivate them
Search providers by name, type, text location or GPS distance	Add staff members and assign which services each can perform
See live availability and book a slot for a service	Generate availability slots in bulk (date range × daily hours × weekdays)
Cancel or reschedule an appointment	See a weekly calendar; cancel, reschedule or complete visits
View a filtered medical timeline and download files	Upload records (vaccination, lab result, prescription...) tied to a visit
Receive instant and scheduled notifications	See a client list built from completed visits

The two user journeys at a glance

Pet parent



Provider

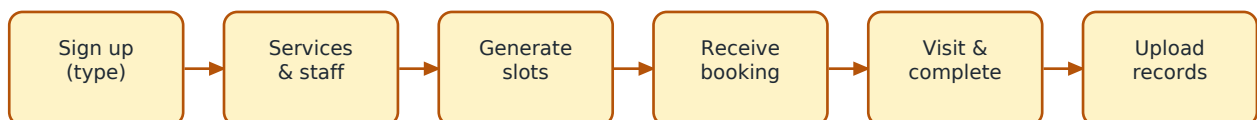


Figure 1 — The booking sits in the middle of both journeys; it is the point where the two sides meet.

2. Technology stack and why each piece was chosen

PawTrack is a classic **three-tier web application**: a browser-based single-page app (presentation tier), a stateless-ish HTTP API (application tier) and a relational database (data tier). Every technology is a mainstream, well-documented choice; the overall philosophy is simple, single-process, no paid services.

Layer	Technology	Role in PawTrack	Why it fits
UI library	React 19	Component-based UI for all pages	Declarative UI; state drives rendering; huge ecosystem
Routing	React Router 7	Client-side routes such as <code>/pets/:petId</code> , <code>/provider/dashboard</code>	Page navigation without full reloads; nested route guards
Styling	Tailwind CSS 4	Utility classes directly in JSX	Fast iteration, consistent design tokens, tiny CSS output
Build/dev	Vite 8	Dev server, hot reload, production bundling, <code>/api</code> proxy	Near-instant startup; proxy removes CORS pain in development
Linting	oxlint	Static checks on frontend code	Very fast Rust-based linter
Runtime	Node.js + TypeScript	Backend language and type system	Same language as frontend; compile-time type safety
HTTP framework	Express 4	Routing, middleware, JSON handling	Minimal, well-understood middleware model
ORM	Prisma 5	Schema, migrations, type-safe queries, transactions	Schema-as-code; generated types catch query mistakes at compile time
Database	SQLite	Single-file relational database	Zero setup; real SQL, foreign keys and transactions
Validation	Zod	Validates every request body and enum value	Runtime checks that also produce TypeScript types
Passwords	bcrypt	Salted, slow password hashing	Industry standard; resistant to brute force
Sessions	jsonwebtoken + cookie-parser	Signed login token in an <code>httpOnly</code> cookie	Stateless auth; no session table needed
Uploads	multer	multipart/form-data parsing and disk storage	Standard Express upload middleware
Scheduling	node-cron	Runs the reminder dispatcher every 5 minutes	In-process cron; no external job queue
Geocoding	OpenStreetMap Nominatim	Converts provider addresses to latitude/longitude	Free, no API key

Key idea: one language end-to-end

JavaScript/TypeScript runs on both the browser and the server. This lowers context-switching, lets the JSON shapes flow naturally between tiers, and means one developer can own a feature end to end.

3. System architecture

The diagram below shows every runtime component and how data flows between them. Notice that there is exactly **one server process** and **one database file**: in-memory structures (the live notification connections) and the scheduled job all live inside that one Node process.

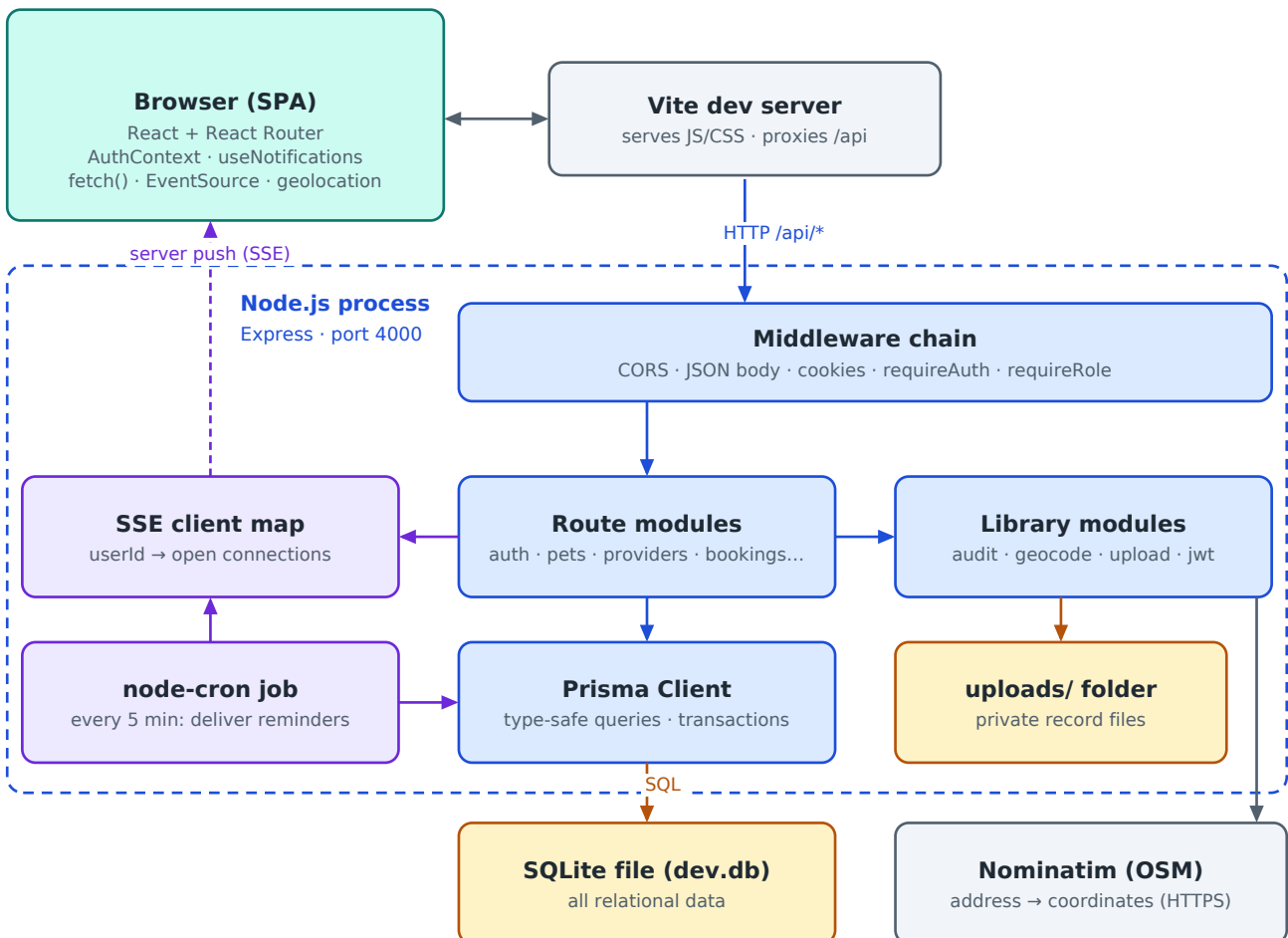


Figure 2 — Runtime architecture. Route handlers call `notifyNow()` and the cron job calls `pushToUser()`; both write to the SSE map, which streams events to open browser tabs (dashed arrow).

Architectural style

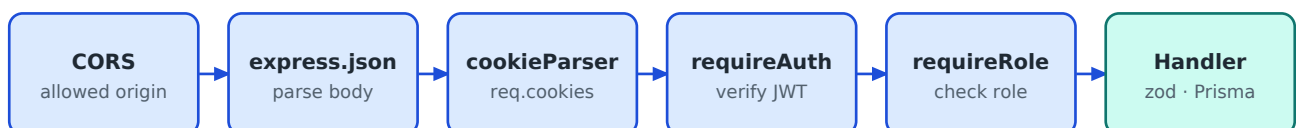
- **Client-server with a JSON REST API.** The React app never touches the database; it only calls `/api/*` endpoints and renders the JSON they return.
- **Single-page application (SPA).** The browser downloads the app once; afterwards navigation happens in JavaScript and only data moves over the network.
- **Layered backend.** Routes (HTTP concerns) → small library modules (auth, audit, notifications, geocoding, uploads) → Prisma (data access) → SQLite (storage).
- **Same-origin in development.** Vite proxies `/api` to port 4000, so the browser sees one origin; cookies "just work" and CORS is only a fallback.
- **Push + pull for notifications.** The browser pulls the list on load and receives new items by server push (Server-Sent Events).

4. Backend structure and the request lifecycle

Path	Responsibility
src/index.ts	Creates the Express app, installs global middleware, mounts routers, installs the error handler, starts listening, starts the cron job.
src/middleware/auth.ts	requireAuth (reads + verifies the JWT cookie) and requireRole(role) (403 if the wrong role).
src/routes/auth.ts	Register, login, logout, current user (me), update profile, delete account.
src/routes/pets.ts	Pet CRUD (soft delete), medical records list/upload/download/delete.
src/routes/providers.ts	Public discovery + availability; provider-only management of services, staff, slots, clients.
src/routes/bookings.ts	Create, list, view, cancel, reschedule, complete bookings.
src/routes/notifications.ts	SSE stream, list notifications, mark as read.
src/routes/reference.ts	Species and breed lookup tables with find-or-create.
src/lib/*	prisma (single DB client), jwt, sse, notifications, cron, audit, geocode, upload, enums, asyncHandler.
prisma/schema.prisma	The single source of truth for the database structure.
prisma/migrations/*	Ordered SQL migration history (init → age/weight → geo → species/breed tables).
prisma/seed.ts	Idempotent seed of starter species and breeds using upsert.

How a request flows through the server

Express processes a request by passing it through an ordered list of **middleware functions**. Each one can inspect or modify the request, end the response, or call `next()` to hand over to the next function. PawTrack's chain looks like this:



Any thrown error / rejected promise → `asyncHandler` → global error handler → 400 (upload errors) or 500

Figure 3 — The middleware pipeline for a protected endpoint.

Three patterns repeated in every handler

- **Validate first.** The body is checked with a Zod schema using `safeParse`. Invalid input returns **400** with the list of issues before any database work happens.
- **Scope every query to the caller.** Instead of "find booking by id", handlers ask "find booking by id where the caller is the parent or the provider". A record that exists but belongs to someone else is indistinguishable from one that does not exist (**404**), which avoids leaking information.
- **Wrap multi-step writes in a transaction.** `prisma.$transaction(async tx => ...)` makes the steps all-or-nothing, and the audit entry and notifications are written inside the same transaction.

Why asyncHandler exists

Express 4 was designed before async/await. If an async handler throws, Express does not notice and the request hangs. The tiny `asyncHandler` wrapper attaches `.catch(next)` so any failure is routed to the central error handler, which logs it and returns a clean JSON error.

```
export function asyncHandler(handler) {  
  return (req, res, next) => { handler(req, res, next).catch(next); };  
}
```

HTTP status codes used consistently

Code	Meaning in PawTrack	Example
200 / 201 / 204	Success / created / success with no body	Booking created → 201; logout → 204
400	Input failed validation or a business rule	Slot in the past; dailyEnd before dailyStart
401	Not logged in or token invalid/expired	No cookie on /api/auth/me
403	Logged in but wrong role	A provider calling POST /api/bookings
404	Not found or not yours	Viewing another parent's pet
409	Conflict with current state	Slot just taken; cancelling a completed visit; duplicate email
500	Unexpected server error	Caught by the global error handler

5. Authentication and authorization

Authentication answers "who are you?"; **authorization** answers "what are you allowed to do?". PawTrack uses stateless token authentication plus two layers of authorization.

5.1 Password storage with bcrypt

Passwords are never stored. At registration the server stores `bcrypt.hash(password, 10)`. `bcrypt` adds a random **salt** (so identical passwords produce different hashes) and is deliberately **slow** (cost factor 10 means 2^{10} internal rounds), making large-scale guessing expensive if the database ever leaks. At login, `bcrypt.compare` re-hashes the attempt with the stored salt and compares. Wrong email and wrong password produce the same message, so attackers cannot probe which emails are registered.

5.2 Sessions with a JWT in an httpOnly cookie

After a successful login or registration the server creates a **JSON Web Token** containing `{ userId, role }`, signed with the server secret (JWT_SECRET) and valid for 7 days. A JWT has three parts — header, payload, signature — and anyone can read the payload, but only the server can produce a valid signature. So the server can trust the payload without a database lookup.

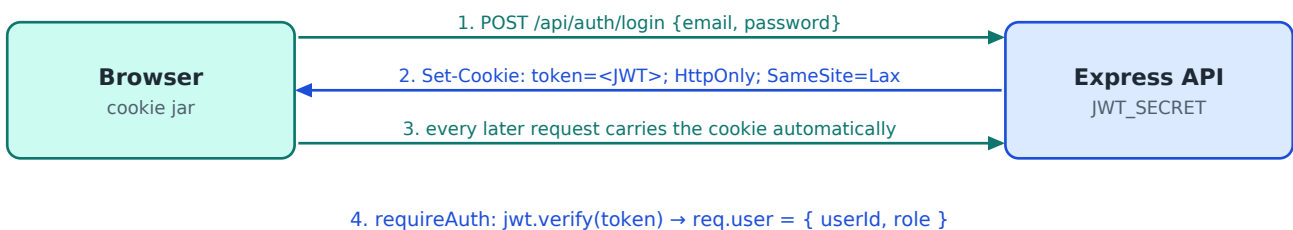


Figure 4 — Login and subsequent authenticated requests.

Cookie option	What it protects against
httpOnly: true	JavaScript cannot read the token, so an XSS bug cannot steal the session.
sameSite: 'lax'	The cookie is not sent on most cross-site requests, reducing CSRF risk.
secure (in production)	Cookie is sent only over HTTPS, preventing interception on the network.
maxAge: 7 days	Matches the JWT expiry; sessions end on their own.

Logout simply clears the cookie. Because the design is stateless there is no server-side session to delete — the trade-off being that a stolen token stays valid until it expires (see section 16).

5.3 Authorization: role, ownership, relationship

- **Role-based (RBAC).** Every user is either PET_PARENT or PROVIDER. `requireRole('PROVIDER')` guards all `/api/providers/me/*` routes; only pet parents can create bookings.
- **Ownership-based.** Queries include the caller's id: a parent may edit only pets where `ownerId = me`; a provider may edit only services/staff where `providerId = me`.

- **Relationship-based.** A provider may read a pet and its records only if a booking exists between that provider and that pet (`loadAccessiblePet`). The care relationship grants access; write access to the pet itself stays with the owner.

5.4 The frontend side of auth

On startup `AuthContext` calls `GET /api/auth/me`. If the cookie is valid, the user and profile are stored in React context; if it returns 401 the user is treated as logged out.

`ProtectedRoute` redirects anonymous visitors to `/login` and sends a user with the wrong role to their own home page; `PublicOnlyRoute` keeps logged-in users away from the signup/login screens.

Important principle

Frontend route guards are only for user experience. Real security is enforced on the server — every API route checks the cookie, role and ownership independently, so bypassing the UI achieves nothing.

6. The data model

The database has 15 tables defined in `schema.prisma`. They fall into five groups: identity, pets & health, provider catalogue, scheduling & booking, and cross-cutting (notifications, audit, and reserved AI tables).

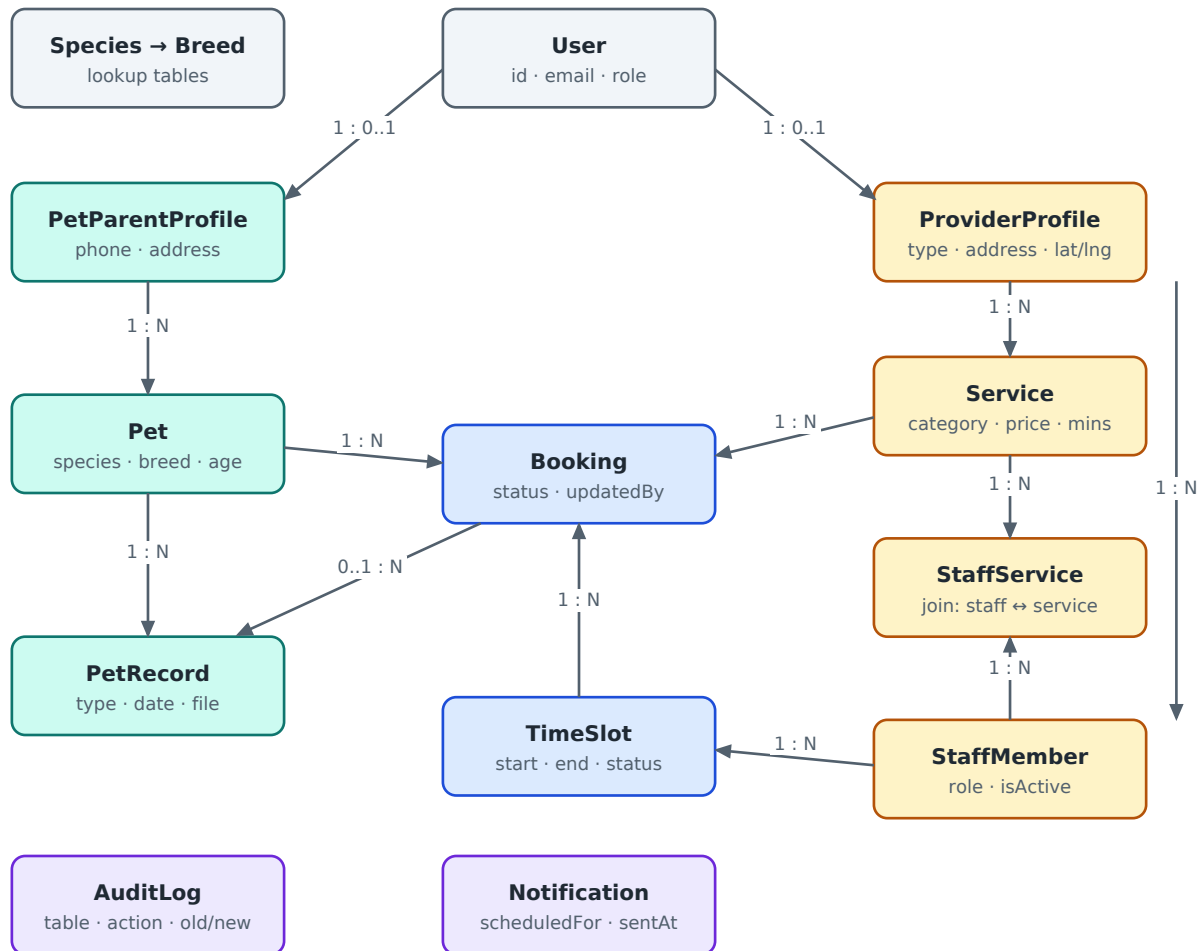


Figure 5 — Simplified entity-relationship view (arrows point from the "one" side to the "many" side). Booking also references PetParentProfile, ProviderProfile and StaffMember directly; Service ↔ StaffMember is N : M via StaffService.

6.1 Key relationships explained

- **User → profile (1 : 0..1).** A single users table holds login data for everyone; role-specific data lives in PetParentProfile or ProviderProfile, whose primary key is the user id (a shared-primary-key one-to-one).
- **Staff ↔ Service (N : M).** A groomer might do baths and nail trims; a nail trim might be done by several staff. The join table StaffService with composite key (staffId, serviceId) models this "who is qualified for what".
- **TimeSlot belongs to a StaffMember, not a Service.** Availability is a property of a person's time; any service that person is qualified for can be booked into it.
- **TimeSlot → Booking is 1 : N on purpose.** When a booking is cancelled its slot is freed and can be booked again, so a slot accumulates booking history. "At most one active booking per slot" is enforced by the slot's status, not by a unique constraint.
- **PetRecord → Booking is optional.** Parents can add records themselves (no booking), while providers attach records to the specific visit they came from.

6.2 Design decisions worth knowing

Decision	Explanation
UUID primary keys	IDs are random UUIDs, not 1, 2, 3... — they cannot be guessed or enumerated and can be generated without asking the database.
Enums as validated strings	SQLite has no ENUM type, so columns like status/role are strings and the allowed values are enforced by Zod enums in <code>lib/enums.ts</code> .
snake_case in DB, camelCase in code	@map/@@map keep SQL conventional while TypeScript stays idiomatic.
Soft deletes	Pets and records get a <code>deletedAt</code> timestamp instead of being erased, so medical and booking history survives; all reads filter <code>deletedAt: null</code> .
Selective cascades	Deleting a user cascades to profile, pets, services, staff. Bookings and audit entries deliberately do not cascade — history should not silently vanish.
Indexes on access paths	e.g. (<code>providerId</code> , <code>status</code>) on bookings and (<code>staffId</code> , <code>status</code> , <code>startDatetime</code>) on slots match the exact filters the dashboards use.
Unique (<code>staffId</code> , <code>startDatetime</code>)	A staff member can never have two slots starting at the same moment, which also makes bulk slot generation idempotent.
JSON stored as text	Notification payloads and audit old/new values are JSON strings — flexible, schema-less details alongside strongly typed columns.
Migrations	Every schema change is an ordered SQL migration, so any environment can be rebuilt to the exact same structure.

7. The booking engine (concurrency and state)

Booking is the most important write in the system and the one most exposed to a classic concurrency bug: **two people booking the same slot at the same moment**. PawTrack solves this with an atomic compare-and-set inside a database transaction.

7.1 The steps of POST /api/bookings

#	Step	Failure response
1	Validate body {petId, serviceId, staffId, slotId} with Zod	400
2	Pet exists, is not deleted, and belongs to the caller	404
3	Service exists and is active	404
4	Staff member exists, is active, and works for that service's provider	404
5	Staff member is qualified for the service (StaffService row exists)	400
6	Slot exists for that staff member and is not in the past	404 / 400
7	Atomically flip the slot AVAILABLE → BOOKED (see below)	409 if someone else won
8	Create the booking row (status PENDING)	—
9	Write an audit log entry	—
10	Schedule four reminders (parent + provider × day-of 8am + 2 h before)	—
11	Notify the provider instantly ("New booking: Bruno for Grooming")	—

All eleven steps run inside one \$transaction. If anything fails, everything is rolled back — there can never be a booked slot with no booking, or a booking with no reminders.

7.2 The double-booking guard

```
const locked = await tx.timeSlot.updateMany({
  where: { id: slotId, status: "AVAILABLE" }, // condition
  data: { status: "BOOKED" },               // change
});
if (locked.count === 0) throw new BookingError(409, "This slot was just booked by someone else");
```

The trick is that the check and the change happen in **one SQL statement**: `UPDATE time_slots SET status='BOOKED' WHERE id=? AND status='AVAILABLE'`. The database executes writes to a row one at a time, so if two requests race, the first changes one row; by the time the second runs, the status is already BOOKED, its WHERE clause matches nothing, and it gets `count = 0`.

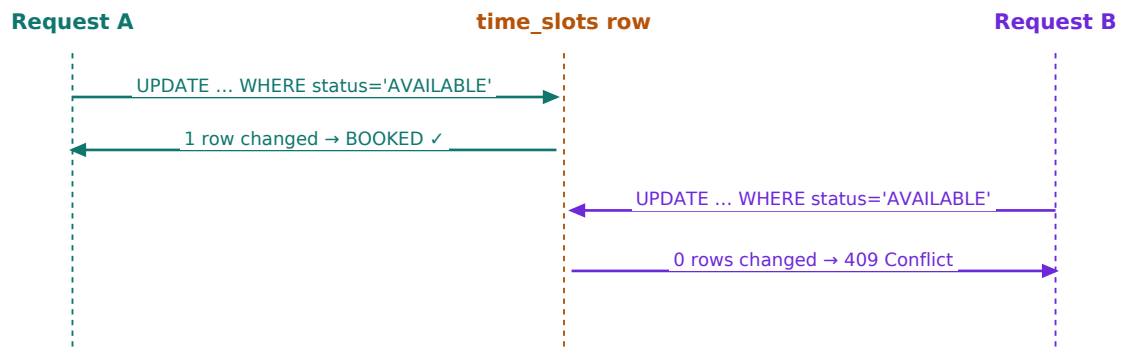


Figure 6 — Two simultaneous booking attempts: exactly one wins, the other receives a clear 409.

Why not "read, check, then write"?

If the code first read the slot, saw `AVAILABLE`, and then wrote `BOOKED`, both racing requests could read `AVAILABLE` before either wrote — the classic **check-then-act race condition**. Folding the check into the write's `WHERE` clause (optimistic concurrency / compare-and-set) removes the gap entirely. Rescheduling uses the same guard on the new slot before releasing the old one.

7.3 Booking lifecycle (state machine)

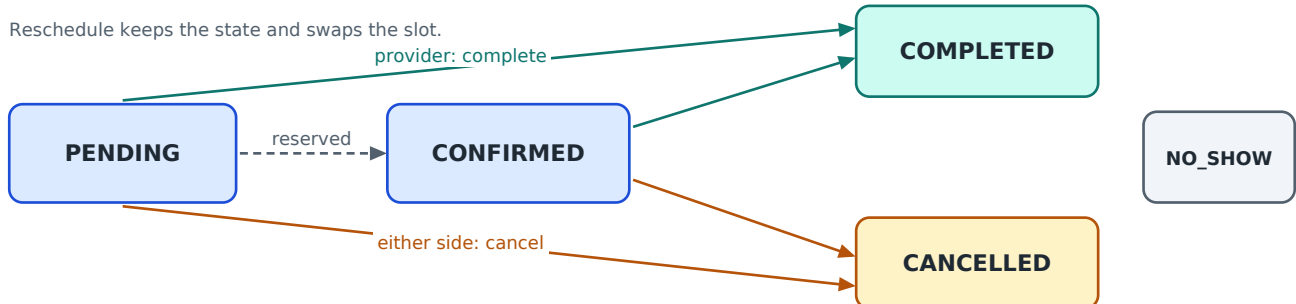


Figure 7 — Booking states. Cancel and reschedule are allowed only from `PENDING` or `CONFIRMED`; `COMPLETED` and `CANCELLED` are terminal.

- **Cancel** (either party): slot goes back to `AVAILABLE`, booking becomes `CANCELLED`, `updatedByUserId` records who did it, and the other party is notified instantly.
- **Reschedule** (either party): lock new slot → free old slot → point booking at the new slot → audit → notify.
- **Complete** (provider only): marks the visit done. Completed visits are what populate the provider's client list, and records can only be attached to visits whose start time has passed and are not cancelled.
- **Deleting a pet** cancels its active bookings and frees their slots in the same transaction, so no orphaned appointments are left behind.

8. Availability: generating time slots

Providers do not create appointments one by one. They describe a **working pattern** and the server expands it into concrete slots. The request looks like:

```
POST /api/providers/me/staff/:staffId/slots
{ "startDate": "2026-10-05", "endDate": "2026-10-09",
  "dailyStart": "09:00", "dailyEnd": "13:00",
  "slotMinutes": 30, "daysOfWeek": [1,2,3,4,5] } // Mon-Fri
```

The algorithm

- Convert dailyStart and dailyEnd to minutes since midnight (09:00 → 540, 13:00 → 780) and reject if end ≤ start.
- Walk day by day from startDate to endDate. Skip days whose weekday is not in daysOfWeek.
- Inside each allowed day, step from dailyStart in increments of slotMinutes while m + slotMinutes ≤ dailyEnd (so a slot never overruns closing time). Each step is a candidate (start, end).
- Look up which candidate start times already exist for this staff member and **skip them**.
- Insert the rest in a single createMany and report {created, skipped}.

With the example above: 4 hours ÷ 30 minutes = 8 slots/day × 5 days = **40 slots**. Running the same request again creates 0 and skips 40 — the operation is **idempotent**, which makes it safe to retry or to extend a schedule with overlapping ranges.

Availability as seen by pet parents

GET /api/providers/:providerId/services/:serviceId/slots finds every active staff member qualified for the service, then returns their future slots with status AVAILABLE, sorted by time and labelled with the staff name. Removing a slot is only allowed while it is still AVAILABLE; booked history is protected.

Effective activity of a service

A service can be switched on but have no active staff able to perform it, which would make it unbookable. The API computes effectiveActive = isActive AND at least one qualified active staff and shows that to the provider, without altering the stored flag — so re-assigning staff brings the service back automatically.

9. Notifications: scheduled reminders and real-time push

PawTrack has two kinds of notifications that share one table and one delivery channel:

	Instant notifications	Scheduled reminders
Created by	notifyNow()	scheduleAppointmentReminders()
Triggered by	New booking, cancel, reschedule, provider uploads a record	Creating a booking (four rows at once)
sentAt when created	Now	null (hidden) — scheduledFor set to the future

	Instant notifications	Scheduled reminders
Becomes visible	Immediately	When the cron job finds scheduledFor ≤ now and sets sentAt
Pushed live?	Yes, from the request that caused it	Yes, by the cron job when it delivers

The core idea: sentAt is the visibility switch

GET /api/notifications only returns rows with sentAt set. A reminder for next Tuesday therefore exists in the database from the moment of booking but is invisible until it is due. Every 5 minutes, node-cron runs:

```
due = notifications WHERE scheduledFor <= now AND sentAt IS NULL
for each: set sentAt = now, then pushToUser(userId, notification)
```

This is a simple, database-backed **job queue**: the table is the queue, scheduledFor is the due time and sentAt marks a job as done. Because state lives in the database, a server restart loses nothing — overdue reminders are simply delivered on the next tick. The granularity is up to 5 minutes, which is fine for "day of" and "2 hours before" reminders.

Server-Sent Events (SSE)

To show a notification without the user refreshing, the server must be able to send data to the browser. PawTrack uses **Server-Sent Events**: the browser opens new EventSource('/api/notifications/stream'), and the server keeps that HTTP response open forever, writing data: {json}\n\n whenever there is news.



Figure 8 — Real-time delivery. A heartbeat comment (":") is written every 20 s to keep proxies from closing idle connections; on disconnect the response is removed from the map.

- **Why SSE instead of WebSockets?** Traffic is one-way (server → browser), SSE is plain HTTP (works with the existing cookie auth and the Vite proxy), and EventSource reconnects automatically.
- **Why a Set per user?** One person may have several tabs open; each gets the event.
- **Resilience.** If a push is missed (tab closed, connection dropped), nothing is lost: the notification row is already in the database and appears on the next page load via the normal GET.
- **Frontend.** useNotifications() loads the latest 50, subscribes to the stream, prepends new events, computes the unread count for NotificationBell, and marks items read optimistically (UI first, then PATCH /:id/read).

10. Provider discovery: geocoding and distance

To answer "which groomers are near me?" the system needs coordinates for both the provider and the searcher.

10.1 Geocoding provider addresses

Providers enter their address as separate fields — street, city, state, postal code. Geocoding (turning an address into latitude/longitude) is done with OpenStreetMap's free **Nominatim** service using its structured query parameters, which is more accurate than a single free-text line because the service does not have to guess which part is the city and which is the street.

- **Graceful fallback.** Structured search is strict — a misspelt or poorly mapped street returns nothing. So the code retries without the street, landing at least on the city/postal area.
- **Best-effort, outside the transaction.** Geocoding runs after the account is saved. A slow or failed external call never blocks sign-up; the provider simply has no coordinates and sorts last.
- **Re-geocode on change.** Updating any address field through PATCH /api/auth/me recomputes (or clears) the coordinates.
- **Storage.** Only the joined address string and the lat/lng are stored; the structured parts exist just long enough to be sent to the geocoder.

10.2 The searcher's position

On the Find-a-Provider page, "Use my location" calls the browser's `navigator.geolocation` API (the user must grant permission) and sends `lat/lng` as query parameters. Alternatively the user can type a location, which filters providers whose address text contains it.

10.3 Distance with the Haversine formula

The Earth is (approximately) a sphere, so the straight-line "flat map" distance is wrong over any real distance. The **Haversine formula** gives the great-circle distance — the shortest path along the surface:

$$a = \sin^2(\Delta\phi/2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2(\Delta\lambda/2)$$

$$d = 2 \cdot R \cdot \text{asin}(\sqrt{a}) \quad \phi = \text{latitude}, \lambda = \text{longitude (radians)}, R = 6371 \text{ km}$$

For each provider with coordinates, the API computes `distanceKm` and sorts ascending. Providers without coordinates are not dropped — they sort to the end, so they remain discoverable. Search by name and filter by provider type are applied before ranking.

Design trade-off

Distances are computed in application code over all matching providers. That is simple and exact for a small dataset. At large scale you would pre-filter with a bounding box on indexed `lat/lng` columns, or use a spatial database (e.g. PostGIS) — see section 16.

11. Medical records and secure file handling

A pet's medical history is a list of PetRecord rows — each with a type (VACCINATION, DEWORMING, LAB_RESULT, PRESCRIPTION, VISIT_SUMMARY, IMAGING, OTHER), title, description, date and an optional attached file.

11.1 Uploading

- The browser sends multipart/form-data (the `api.upload` helper deliberately omits the JSON Content-Type so the browser can set the multipart boundary).
- **multer** validates the file before saving: only PNG, JPEG, WebP, HEIC or PDF, maximum 15 MB. Bad files produce a 400 via the global error handler.
- Files are stored as `uploads/<petId>/<random-uuid>.<ext>`. The random name prevents collisions, path tricks and guessable URLs; the original filename is not trusted.
- When a **provider** uploads, the record may be tied to a booking — but only their own, not cancelled, and not before the appointment starts — and the owner is notified instantly.

11.2 Downloading: no public URLs

The `uploads/` directory is **not** served statically. The database stores only the internal filename; the API exposes `fileUrl` as `/api/pets/records/:id/file`. That route re-checks access (owner, or provider with a booking for that pet) on every request before streaming the file. Health records are sensitive, so there is deliberately no way to reach a file without passing authorization.

11.3 Browsing the timeline

Records can be filtered by category and time: presets (3 months, 6 months, 1 year, all) or an explicit from/to range, sorted newest first. Deleting a record is a soft delete.

12. Reference data: species and breeds

Free-text fields drift: "dog", "Dog", "DOG", "doggo". To keep data clean without being restrictive, Species and Breed are lookup tables that power the SpeciesBreedPicker dropdowns.

- **Seeded** with common species and breeds by an idempotent upsert script (safe to run many times).
- **Find-or-create.** If a user types a species/breed not in the list, POST `/species` or `/breeds` first looks for a case-insensitive match and returns the existing canonical entry; only a genuinely new name is inserted.
- **Case-insensitivity in code.** SQLite's Prisma connector lacks case-insensitive matching, so the comparison is done in JavaScript — acceptable because the tables are small.
- **Public on purpose.** These endpoints need no login because the pet-parent sign-up form uses them before an account exists; the data is non-sensitive shared vocabulary.
- **Loose coupling.** `Pet.species/breed` remain plain strings (not foreign keys); the lookup tables govern input, not storage.

13. Frontend architecture

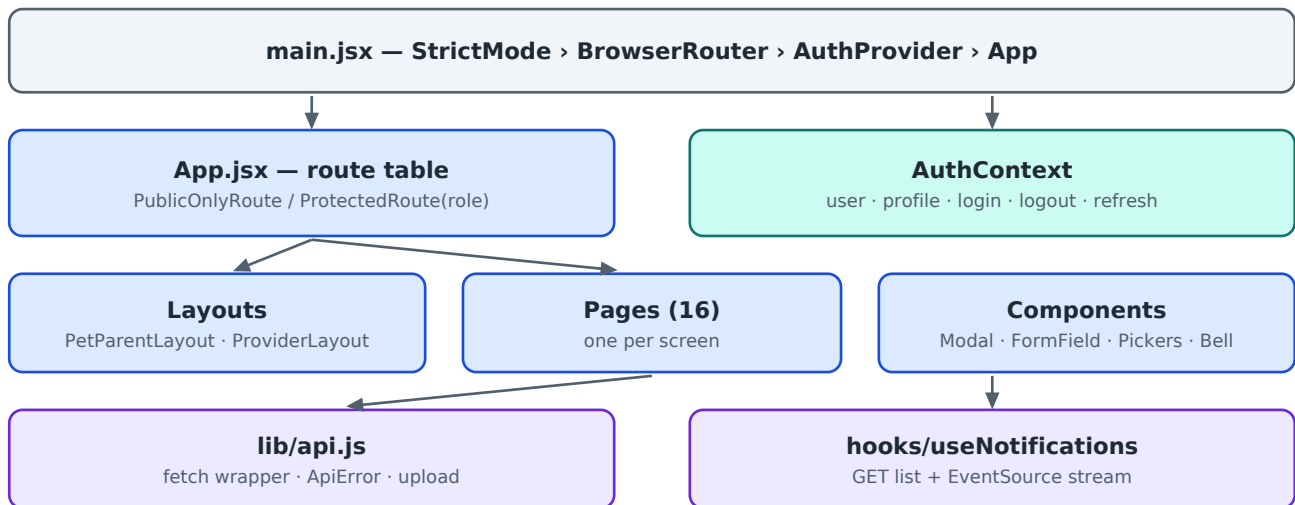


Figure 9 — Frontend composition.

Routes

Area	Routes
Public only	/login, /signup (role select), /signup/pet-parent, /signup/provider
Pet parent	/ (home, pets), /account, /pets/:petId (profile + records), /providers (search), /providers/:providerId (services + booking), /bookings, /bookings/:bookingId
Provider	/provider/dashboard (weekly calendar), /provider/services (services, staff, slots), /provider/settings, /provider/appointments/:bookingId, /provider/clients
Fallback	* → redirect to /

Key concepts

- **Global state via Context.** Only the session is global (**AuthContext**). Everything else is local page state fetched on mount — a deliberate choice that keeps data fresh and avoids a heavyweight state library.
- **A single API wrapper.** **lib/api.js** prefixes **/api**, sends cookies (**credentials: 'include'**), serialises JSON, handles 204, and converts error responses into an **ApiError** carrying the HTTP status and the server's message — so every page handles errors the same way.
- **Custom hooks** encapsulate reusable behaviour (e.g. notifications) so components stay presentational.
- **Role-specific layouts** provide each side's navigation shell; pages render inside them.
- **Reusable form primitives** (**FormField**, **AgeInput**, **SpeciesBreedPicker**, **UploadRecordForm**, **RescheduleModal**, **ConfirmDialog**) keep behaviour and styling consistent.
- **Utility-first styling** with Tailwind and a design-token palette (surface, on-surface, outline...) gives a consistent look and responsive layouts.
- **App icons.** 192/512 px and Apple touch icons ship in **public/**, a starting point for making the app installable as a PWA (a web manifest and service worker would complete it).

14. Data integrity, auditing and security principles

14.1 Audit trail

Important changes — bookings created/cancelled/rescheduled/completed, services and staff edited or deleted, pets deleted, profiles updated, accounts deleted — write an `AuditLog` row with the table, record id, action (INSERT/UPDATE/DELETE), who did it, when, and JSON snapshots of old and new values. Because it is written in the **same transaction** as the change, the log can never disagree with the data. When a user deletes their account, their audit entries are kept but anonymised (changedBy set to null).

14.2 Protecting history

- Accounts with any booking history cannot be deleted (409) — the other party's records depend on them.
- Staff with booking history cannot be deleted, only deactivated.
- Only AVAILABLE slots can be removed; cancelled/completed bookings cannot be changed again.
- Pets and records are soft-deleted.

14.3 Security checklist

Threat	Mitigation in PawTrack
Password database leak	bcrypt salted hashing (cost 10)
Session theft via XSS	httpOnly cookie; React escapes rendered text by default
Cross-site request forgery	SameSite=Lax cookie; CORS limited to the configured origin with credentials
Accessing others' data (IDOR)	Every query scoped to the caller; 404 for "not yours"
Privilege escalation	requireRole on role-specific routes; role is inside the signed JWT
Malformed / malicious input	Zod validation on every body; Prisma parameterises all SQL (no injection)
Leaking password hashes	Explicit select of safe user fields in included relations
Malicious uploads	MIME whitelist, 15 MB limit, random filenames, non-public folder, auth-checked download
Race conditions	Transactions and conditional updates (compare-and-set)
Account enumeration	Identical error for unknown email and wrong password

15. API reference

All paths are prefixed with /api. **Auth:** Public, Any (logged in), Parent, Provider.

Method & path	Auth	Purpose
GET /health	Public	Liveness check
POST /auth/register	Public	Create parent (+ first pet) or provider; sets cookie
POST /auth/login · POST /auth/logout	Public	Start / end session
GET · PATCH · DELETE /auth/me	Any	Current user + profile / update / delete account
GET · POST /pets	Parent	List / create own pets
GET · PUT · DELETE /pets/:petId	Parent*	View (provider-with-booking may read) / edit / soft-delete
GET · POST /pets/:petId/records	Owner or provider-with-booking	Filtered timeline / upload record (+file)
GET /pets/records/:id/file	Owner or provider-with-booking	Authorised file download
DELETE /pets/records/:id	Parent (owner)	Soft-delete a record
GET /providers	Public	Search: category, search, location, lat, lng
GET /providers/:id	Public	Provider profile and active services
GET /providers/:id/services/:sid/slots	Public	Future available slots for a service
GET · POST /providers/me/services	Provider	List (with effectiveActive) / create
PUT · DELETE /providers/me/services/:id	Provider	Edit / delete service
PATCH /providers/me/services/:id/(activate deactivate)	Provider	Toggle service
GET · POST /providers/me/staff	Provider	List / add staff (with service assignments)
PUT · DELETE /providers/me/staff/:id	Provider	Edit / delete staff (409 if has history)
PATCH /providers/me/staff/:id/(activate deactivate)	Provider	Toggle staff member
GET · POST /providers/me/staff/:id/slots	Provider	List future slots / bulk-generate
DELETE /providers/me/slots/:slotId	Provider	Remove an AVAILABLE slot
GET /providers/me/clients	Provider	Pets seen in completed visits, grouped
GET /bookings?status=	Any	Own bookings (as parent or provider)
POST /bookings	Parent	Create booking (transactional, race-safe)
GET /bookings/:id	Participant	Details incl. owner and visit records
PATCH /bookings/:id/cancel · /reschedule	Participant	Cancel / move to another slot

Method & path	Auth	Purpose
PATCH /bookings/:id/complete	Provider	Mark visit completed
GET /notifications · /notifications/stream	Any	Latest 50 delivered / live SSE stream
PATCH /notifications/:id/read	Any	Mark as read
GET · POST /species; GET /species/:id/breeds; POST /breeds	Public	Lookup + find-or-create

16. Limitations, scaling path and future work

PawTrack is intentionally optimised for simplicity: one process, one file database, no paid services. Being able to name the trade-offs — and how you would lift them — is part of understanding the design.

Current choice	Limitation	How it would scale
SQLite single file	One writer at a time; lives on one machine	PostgreSQL (Prisma makes the switch mostly a config change; enums could become native)
In-memory SSE map	Works only with one server instance	Redis pub/sub or a message broker to fan out across instances
In-process node-cron	Duplicated if several instances run; 5-min granularity	Dedicated worker / job queue (e.g. BullMQ) with locking
Local uploads/ folder	Tied to one disk; no backups/CDN	Object storage (e.g. S3) with short-lived signed URLs
Stateless 7-day JWT	Cannot revoke a stolen token early	Short-lived access token + refresh token, or a server-side session/deny list
Distance computed in JS	Scans all providers	Bounding-box pre-filter on indexed columns, or PostGIS
Local time for slots/reminders	Server timezone determines "8 am"	Store provider timezone; compute in UTC
Public Nominatim	Rate-limited, best-effort	Cache results or use a commercial geocoder
No automated tests yet	Regressions caught manually	Unit tests for lib/, integration tests per route (e.g. concurrent booking test)

Visible hooks for future features

- **AI assistant** — AiConversation and AiMessage tables already exist (a conversation per parent, optionally about a specific pet), ready for a chat-style pet-care assistant.
- **Booking confirmation step** — the CONFIRMED and NO_SHOW states are modelled but not yet driven by an endpoint.
- **Community features** — a COMMUNITY_REPLY notification type is reserved.
- **BLOCKED slots** — a slot status reserved for breaks/holidays.

17. Explaining PawTrack: common questions and model answers

Q. Describe PawTrack's architecture in two sentences.

It is a React single-page app that talks to a Node/Express REST API over JSON, and the API uses Prisma to read and write a SQLite database. Real-time notifications are pushed from the server with Server-Sent Events, and a cron job inside the same process delivers scheduled reminders.

Q. How do you prevent two users from booking the same slot?

Inside a transaction, the slot is claimed with one conditional UPDATE: set status BOOKED where id matches and status is AVAILABLE. The database applies it atomically, so only one request changes the row; the other sees zero affected rows and gets a 409. There is no gap between checking and writing.

Q. How does login work and why use a cookie?

The password is checked with bcrypt, then the server signs a JWT containing the user id and role and sets it as an httpOnly, SameSite=Lax cookie. The browser sends it automatically, the server verifies the signature on each request, and JavaScript can't read it, which protects the session against XSS.

Q. What is the difference between authentication and authorization here?

Authentication is requireAuth verifying the JWT. Authorization is layered: requireRole for role, queries scoped to the caller for ownership, and booking-based access for providers reading a pet's records.

Q. How do reminders get sent at the right time?

Four reminder rows are created when the booking is made, with scheduledFor set and sentAt empty. A node-cron job runs every five minutes, finds due rows, sets sentAt and pushes them over SSE. The list endpoint only shows rows with sentAt, so reminders appear exactly when due and survive restarts.

Q. Why SSE and not WebSockets or polling?

Updates only flow server-to-client, so SSE is sufficient; it is plain HTTP that reuses cookie auth, reconnects automatically, and avoids the wasted requests of polling.

Q. How is 'providers near me' calculated?

Provider addresses are geocoded to coordinates with Nominatim at signup or update. The browser supplies the user's coordinates, and the server computes great-circle distance with the Haversine formula and sorts by it.

Q. How are medical files kept private?

They are stored outside any public folder with random names, and the only way to fetch one is an API route that re-checks the requester is the owner or a provider who has a booking with that pet.

Q. Why soft deletes?

Medical and booking history is valuable and referenced by other people's data. Marking rows deleted hides them from normal queries without destroying history or breaking relationships.

Q. What is a transaction and where do you use it?

A group of writes that succeed or fail together (atomicity). PawTrack uses them for registration (user + profile + pet), bookings (slot + booking + audit + reminders + notification), cancel/reschedule, pet deletion and account deletion.

Q. Why is TimeSlot linked to a staff member rather than a service?

Availability belongs to a person's time. Any service that person is qualified for (via the StaffService join table) can use their slot, which avoids duplicating calendars per service.

Q. What would you change for production scale?

Move to PostgreSQL, store files in object storage, replace the in-memory SSE map with Redis pub/sub, run reminders in a separate worker, add refresh tokens, handle timezones explicitly and add automated tests.

Glossary

Term	Meaning
SPA	Single-page application — the page loads once and JavaScript swaps views.
REST API	HTTP endpoints organised around resources (pets, bookings) using GET/POST/PUT/PATCH/DELETE.
Middleware	A function in Express's request pipeline that can inspect, modify or end a request.
ORM	Object-relational mapper — lets code query the database with objects instead of raw SQL (Prisma).
Migration	A versioned script that changes the database schema in a repeatable way.
JWT	JSON Web Token — a signed, self-contained token proving identity.
bcrypt / salt	A slow password-hashing algorithm / random data mixed in so equal passwords hash differently.
Transaction (ACID)	All-or-nothing group of database operations.
Race condition	A bug where the result depends on the timing of concurrent operations.
Compare-and-set	Update a value only if it still has the expected value, in one atomic step.
Idempotent	Doing an operation twice has the same effect as doing it once.
SSE	Server-Sent Events — a one-way, long-lived HTTP stream from server to browser.
Geocoding	Converting an address into latitude/longitude.
Haversine	Formula for distance between two points on a sphere.
Soft delete	Marking a row as deleted with a timestamp instead of removing it.
IDOR	Insecure direct object reference — accessing someone else's data by changing an id.